REST API Integration

Prepared with AI Aid.

Agenda

2.1
Understanding APIs — The Restaurant Analogy

2.2
Real Example: GitHub API — Making Your First Request

2.3
Handling Authentication — API Keys & .env Files

2.4
Pagination — Fetching Large Datasets

2.5
Rate Limiting & Retry Logic with Exponential Backoff

2.6
Error Handling & Logging Best Practices

2.7
Building a Reusable API Client (OOP)

2.1 What is an API?

API = Application Programming Interface

An API lets two programs talk to each other — without revealing how they work internally.

🍳	📋	👨‍🍳	📦	🍽️
Kitchen = Server & Database	**Menu = API Documentation**	**Waiter = API Endpoints**	**Your Order = HTTP Request**	**Your Food = HTTP Response**

Key Insight: You never access the database directly. You ask the API (waiter), and it fetches data for you.

REST (Representational State Transfer) — a set of rules for building APIs over HTTP.

💡 *Pro Tip: Always read the API documentation first. It tells you endpoints, required parameters, and rate limits.*

2.1 HTTP Methods — CRUD Operations

CRUD = Create | Read | Update | Delete — the four fundamental data operations

GET	Retrieve data (read-only)	*"Show me the menu"*
POST	Create new resource	*"I want to place this order"*
PUT	Replace entire resource	*"Change my entire order"*
PATCH	Update part of a resource	*"Just add extra cheese"*
DELETE	Remove a resource	*"Cancel my order"*

📌 *Data scientists mostly use GET (fetch data) and occasionally POST (submit data to models/APIs).*

2.1 HTTP Status Codes — What Did the Server Say?

200 — **OK**
Here's your data!

201 — **Created**
Resource created successfully

400 — **Bad Request**
Your request has an error (check params)

401 — **Unauthorized**
Authentication required (check API key)

403 — **Forbidden**
You don't have permission

404 — **Not Found**
That resource doesn't exist

429 — **Too Many Requests**
Slow down! You hit the rate limit

500 — **Server Error**
The server crashed (not your fault)

⭐ **Pattern: 2xx = Success | 4xx = Client Error (your fault) | 5xx = Server Error (their fault)**

2.2 Real Example: GitHub API — First Request

GitHub offers free public APIs. No signup needed for basic access (60 req/hour unauthenticated).

Install the requests library:

```
pip install requests
```

Basic GET request:

```
import requests

url = 'https://api.github.com/repos/pandas-dev/pandas'

response = requests.get(url)            # Makes HTTP GET request
                                        # returns a Response object
print(response.status_code)             # → 200 (OK)
data = response.json()                  # Parses JSON body → Python dict
print(data['name'])                     # → 'pandas'
print(data['stargazers_count'])         # → 43256
```

💡 *requests.get(url) returns a Response object with .status_code, .json(), .text, .headers*

⚠️ *Pitfall: Always check status_code before calling .json() — a 404 response has no JSON data!*

2.2 Function: get_repo_info()

```python
def get_repo_info(owner, repo):
    """
    Fetch information about a GitHub repository.
    Args:          owner (str): Repository owner (e.g., 'pandas-dev’)
                    repo  (str): Repository name  (e.g., 'pandas')
    Returns:       dict: Repository info, or None if request fails
    """
    url = f'https://api.github.com/repos/{owner}/{repo}’
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()          # dict with 100s of fields
        return {
            'name'        : data['name'],
            'stars'       : data['stargazers_count'],
            'forks'       : data['forks_count’],
            'language'    : data['language'],
            'license'     : data['license']['name'] if data['license'] else 'No license',
        }
    else:
        print(f"Error {response.status_code}")
        return None
info = get_repo_info('pandas-dev', 'pandas')
print(info)
```

2.2 Understanding the HTTP Response Structure

Every HTTP response has 3 parts:

```
# 1. STATUS LINE
HTTP/1.1 200 OK

# 2. HEADERS  (response.headers)
Content-Type: application/json
X-RateLimit-Limit: 60        ← Max requests per hour
X-RateLimit-Remaining: 59    ← Requests left in current window
X-RateLimit-Reset: 1710000000  ← Unix timestamp when limit resets

# 3. BODY  (response.json())
{
    "name": "pandas",
    "stargazers_count": 43256,
     ...
}

# Accessing parts in Python:
print(response.status_code)          # 200
print(response.headers['Content-Type'])  # application/json
data = response.json()                   # Python dict
print(data['name'])                      # pandas
```

💡 *Tip: response.headers acts like a dictionary — use it to monitor rate limits proactively.*

2.3 Authentication — Why APIs Need to Know Who You Are

Without authentication: 60 requests/hour | With authentication: 5,000 requests/hour

1. API Key in Header (Most Common)

2. API Key in Query Params

3. OAuth (Advanced — not needed for data collection)

Why authenticate?

- Track usage per developer
- Prevent abuse of shared infrastructure
- Enable personalized data (your repos, your activity)
- Higher rate limits (5,000 vs 60 req/hr on GitHub)

⚠️ **NEVER hardcode your API key in code! Anyone who reads your code (e.g., on GitHub) can use it.**

2.3 Storing Secrets Safely: .env Files

Step 1 — Create a .env file (never commit this!):

```
# .env  (lives in your project root)
GITHUB_TOKEN=ghp_your_actual_token_here
OPENWEATHER_KEY=your_openweather_key_here
```

Step 2 — Add .env to .gitignore:

```
# .gitignore
.env
*.env
```

Step 3 — Load in Python:

```python
import os
from dotenv import load_dotenv     # pip install python-dotenv

load_dotenv()                       # reads .env into environment

api_key = os.getenv('GITHUB_TOKEN') # retrieves the value
# os.getenv returns None if key missing (safe — no crash)

headers = {
    'Authorization': f'Bearer {api_key}',
    'Accept': 'application/vnd.github.v3+json',
    'User-Agent': 'MyApp/1.0'       # identify your app (required by GitHub)
}
response = requests.get(url, headers=headers)
```

2.3 Query Parameters — Filtering & Customizing Requests

Query parameters are key-value pairs appended to a URL after '?' — they filter or configure results.

```
# URL with query parameters (manually built):
https://api.openweathermap.org/data/2.5/weather?q=Cairo&appid=KEY&units=metric
#  ↑ base URL                                   ↑ q=Cairo  ↑ appid ↑ units

# Better: let requests build it for you (handles URL-encoding)
params = {
    'q'     : 'Cairo',       # City name to search
    'appid': api_key,        # Your API key
    'units': 'metric'        # Temperature unit: 'metric'=°C, 'imperial'=°F
}
response = requests.get('https://api.openweathermap.org/data/2.5/weather',
                        params=params)

# requests automatically encodes to:
# ?q=Cairo&appid=KEY&units=metric

data = response.json()
print(f"Temperature in Cairo: {data['main']['temp']}°C")
print(f"Humidity: {data['main']['humidity']}%")
```

⭐ *Pro Tip: Always use the params={} dict — never concatenate strings! It handles spaces and special characters.*

2.4 Pagination — Handling Large Datasets

Problem: An API has 10,000 results but only returns 100 per request.

Page 1 (items 1-100)	Page 2 (items 101-200)	Page 3 (items 201-300)	... (page N)

Pagination Strategies:

Page-based	`?page=1&per_page=100 → ?page=2&per_page=100`	*Used by: GitHub, Reddit*
Offset-based	`?offset=0&limit=100 → ?offset=100&limit=100`	*Used by: Many REST APIs*
Cursor-based	`?cursor=abc123 (server returns next_cursor)`	*Used by: Twitter/X, Shopify*

⭐ *Cursor-based is most efficient for large datasets — no duplicate items when data changes between pages.*

2.4 Pagination Code — Fetching All Pages

```python
import time

def get_all_repos(org_name, max_pages=None):
    """
    Fetch all repos for a GitHub organization using pagination.
    Args:
        org_name  (str): GitHub organization name (e.g., 'pandas-dev')
        max_pages (int): Stop after N pages (None = fetch all)
    Returns:
        list: All repository dicts
    """
```

Usage:

```python
repos = get_all_repos('pandas-dev', max_pages=3)
print(f"Fetched {len(repos)} repositories")
```

2.4 Pagination Code — Fetching All Pages

```python
def get_all_repos(org_name, max_pages=None):
    all_repos = []
    page = 1
    while True:
        if max_pages and page > max_pages:
            break                          # Safety limit reached
        params = {
            'page'    : page,              # Current page number (1-indexed)
            'per_page': 100                # Items per page (GitHub max = 100)
        }
        url = f'https://api.github.com/orgs/{org_name}/repos'
        response = requests.get(url, params=params)
        if response.status_code != 200:
            print(f"Error: {response.status_code}")
            break

        data = response.json()             # List of repo dicts for this page
        if not data:                       # Empty list = no more pages
            break
        all_repos.extend(data)             # .extend() adds all items (not nested)
        page += 1
        time.sleep(0.5)                    # ← Be polite: 0.5s pause between requests
    return all_repos
```

2.5 Rate Limiting — Why APIs Throttle Requests

Rate limiting = a cap on how many requests you can make in a time window.

😩	GitHub (unauthenticated)	`60 req / hour`
😊	GitHub (authenticated)	`5,000 req / hour`
🌤️	OpenWeather Free	`60 req / minute`
🦜	Twitter/X API Basic	`500,000 tweets / month`

When you exceed the limit, you get HTTP 429 (Too Many Requests). The server sends a Retry-After header.

Checking rate limit from headers:

```
remaining = int(response.headers.get('X-RateLimit-Remaining', 999))
if remaining < 10:
    print("⚠️  Low on API calls! Slow down.")
```

2.5 Building a RateLimiter Class

```python
import time

class RateLimiter:
    """
    Tracks request timestamps and waits if we exceed the limit.
    """
    def __init__(self, max_requests=60, time_window=3600):
        """
        Args:
            max_requests (int): Max requests allowed in the window
            time_window  (int): Window duration in seconds (3600 = 1 hour)
        """
        self.max_requests = max_requests
        self.time_window  = time_window
        self.requests     = []          # Timestamps of past requests
```

2.5 Building a RateLimiter Class

```
    # Inside the class
    def wait_if_needed(self):
        """Sleep if rate limit is reached; then record this request."""
        now = time.time()                # Current Unix timestamp (float)
        # Discard timestamps outside the time window
        self.requests = [t for t in self.requests if now - t < self.time_window]

        if len(self.requests) >= self.max_requests:
            sleep_time = self.time_window - (now - self.requests[0])
            if sleep_time > 0:
                print(f"Rate limit hit. Sleeping {sleep_time:.1f}s...")
                time.sleep(sleep_time)
                self.requests = []

        self.requests.append(now)        # Record this request

# Usage: 10 requests per minute
limiter = RateLimiter(max_requests=10, time_window=60)
for i in range(15):
    limiter.wait_if_needed()            # blocks if needed
    response = requests.get(url)
```

⭐ *Pro Tip: time.time() returns seconds since Unix epoch (Jan 1 1970). Subtracting two values gives elapsed seconds.*

2.5 Exponential Backoff — Retry Without Hammering

Exponential backoff = wait progressively longer after each failure (1s, 2s, 4s, 8s, 16s…)

Try 1	Try 2	Try 3	Try 4	Try 5
Fail → wait 1s	Fail → wait 2s	Fail → wait 4s	Fail → wait 8s	✓ Success!

Pitfall: Without retry logic, a single 500 error will crash your entire data collection script.

2.5 Exponential Backoff — Retry Without Hammering

```python
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry
def create_robust_session():
    """
    Create a requests Session with automatic retry + backoff.
    Returns:
        requests.Session: Configured session
    """
    session = requests.Session()
    retry_strategy = Retry(
        total=5,            # Maximum retry attempts
        backoff_factor=1,   # Wait: 0, 1, 2, 4, 8 seconds (1st retry happens immediately (0s delay))
        # backoff_factor=1 → sleep = backoff_factor * (2 ** (attempt-1))
        status_forcelist=[429, 500, 502, 503, 504],  # Retry on these codes
        allowed_methods=["GET", "POST", "HEAD"]      # Methods to retry
    )
    adapter = HTTPAdapter(max_retries=retry_strategy)
    session.mount("https://", adapter)  # Apply for HTTPS
    session.mount("http://",  adapter)  # Apply for HTTP
    return session

session = create_robust_session()
response = session.get(url)            # Retries automatically on failure!
```

2.6 Error Handling — Never Let Your Script Crash Silently

```python
def fetch_with_error_handling(url, max_retries=3):
    """
    Robust fetch with retry and logging.
    Args:
        url         (str): API endpoint URL
        max_retries (int): Max retry attempts before giving up
    Returns:
        dict | None: Parsed JSON on success, None on failure
    """
```

2.6 Error Handling — Never Let Your Script Crash Silently

```python
for attempt in range(max_retries):
        try:
            response = requests.get(url, timeout=10)  # timeout=10 → fail after 10s
            if response.status_code == 200:
                return response.json()            # ✓ Success
            elif response.status_code == 404:
                return None                       # Resource doesn't exist, don't retry
            elif response.status_code == 429:
                wait = int(response.headers.get('Retry-After', 60))
                time.sleep(wait)                  # Wait as instructed
                continue                          # Retry
            elif response.status_code >= 500:
                time.sleep(2 ** attempt)          # Exponential backoff: 1, 2, 4s
                continue
        except requests.exceptions.Timeout:
            time.sleep(2 ** attempt)              # Network too slow → retry
        except requests.exceptions.ConnectionError:
            time.sleep(2 ** attempt)              # No internet → retry
        except Exception as e:
            print(f"Unexpected: {e}")
            return None
    return None   # Exhausted all retries
```

⭐ *Best Practice: timeout=10 prevents hanging forever on slow servers. Always set a timeout!*

2.6 Logging — Track What Your Script Is Doing

Logging Levels (in order of severity):

DEBUG	INFO	WARNING	ERROR
Detailed diagnostic info	Routine progress updates	Something unexpected but recoverable	A request failed

2.6 Logging — Track What Your Script Is Doing

```python
import logging
logging.basicConfig(       # Configure logging (do this ONCE at the top of your script)
    level=logging.INFO,    # Show INFO and above (DEBUG, INFO, WARNING, ERROR, CRITICAL)
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    # asctime   = human-readable timestamp
    # name      = logger name (usually __name__ = module name)
    # levelname = INFO / WARNING / ERROR
    # message   = your custom message
    handlers=[
        logging.FileHandler('api_requests.log'),   # Save to file
        logging.StreamHandler()                    # Also print to console
    ]
)
logger = logging.getLogger(__name__)  # __name__ = current module name

# Use it:
logger.info("Starting data collection")         # Routine info
logger.warning("Only 5 API calls remaining")    # Something to notice
logger.error("Request failed: status 500")      # Something went wrong
logger.debug("Raw response: " + str(data))      # Detailed debug info
# Log Output in api_requests.log:
# 2024-03-15 10:30:45 - __main__ - INFO     - Starting data collection
# 2024-03-15 10:30:46 - __main__ - INFO     - Fetched 100 repos
# 2024-03-15 10:30:47 - __main__ - WARNING  - Only 5 API calls remaining
```

2.7 Building a Reusable API Client (OOP) — Part 1

Why a class? Encapsulates session, headers, rate limiting, and logging in one reusable object.

```
GitHubAPI
├── __init__()          → Setup everything
├── _create_session()   → Build session with retry logic
├── get()               → Core GET request method
├── get_repo()          → Fetch one repository
├── get_user_repos()    → Fetch all repos for a user
├── search_repos()      → Search with filters
└── to_dataframe()      → Convert results to DataFrame
```

2.7 Building a Reusable API Client (OOP) — Part 1

Why a class? Encapsulates session, headers, rate limiting, and logging in one reusable object.

```
class GitHubAPI:
    """Reusable GitHub API client with rate limiting & logging."""
    def __init__(self, token=None):
        """
        Args:
            token (str): Personal Access Token (optional, increases limits)
        """
        self.base_url    = 'https://api.github.com'
        self.session     = self._create_session()   # Reuse TCP connection
        self.rate_limiter= RateLimiter(max_requests=5000, time_window=3600)
        self.logger      = logging.getLogger(self.__class__.__name__)
        # self.__class__.__name__ = 'GitHubAPI' (auto-naming the logger)
        self.session.headers.update({
            'Accept'    : 'application/vnd.github.v3+json',  # Request API v3
            'User-Agent': 'Library-Tutorial/1.0'              # Required by GitHub
        })
        if token:
            self.session.headers.update({
                'Authorization': f'Bearer {token}'   # f'Bearer ...' = standard format
            })
```

2.7 Building a Reusable API Client (OOP) — Part 1

Why a class? Encapsulates session, headers, rate limiting, and logging in one reusable object.

```
def _create_session(self):
    """Private helper: session + retry. (Leading _ = private convention)"""
    session = requests.Session()
    retry   = Retry(total=3, backoff_factor=1, status_forcelist=[429,500,502,503])
    adapter = HTTPAdapter(max_retries=retry)
    session.mount("http://", adapter)
    session.mount("https://", adapter)
    return session
```

💡 *requests.Session() reuses the same TCP connection → faster & more efficient than individual requests.*

2.7 Reusable API Client — Methods

```python
def get(self, endpoint, params=None):
    """
    Core GET method with rate limiting.
    Args:
        endpoint (str): API path, e.g. '/repos/pandas-dev/pandas'
        params   (dict): Optional query parameters
    Returns:
        dict | list: Parsed JSON
    Raises:
        requests.HTTPError on non-2xx response
    """
    self.rate_limiter.wait_if_needed()       # Pause if limit reached
    url = f"{self.base_url}/{endpoint.lstrip('/')}"
    # .lstrip('/') removes leading slash to avoid double-slash
    try:
        response = self.session.get(url, params=params, timeout=10)
        response.raise_for_status()              # Raises for 4xx/5xx
        self.logger.info(f"GET {endpoint} – Status {response.status_code}")
        return response.json()
    except Exception as e:
        self.logger.error(f"Error fetching {endpoint}: {e}")
        raise
```

⚠️ *raise_for_status() converts HTTP errors (404, 500) into Python exceptions — lets try/except catch them.*

2.7 Reusable API Client — Methods

```python
def get_repo(self, owner, repo):
    """Get info for a single repository."""
    return self.get(f'/repos/{owner}/{repo}')

def get_user_repos(self, username):
    """Get all repositories for a user."""
    return self.get(f'/users/{username}/repos', params={'per_page':100})


def search_repos(self, query, language=None, min_stars=None):
    """Search GitHub repos. Builds GitHub search syntax."""
    q_parts = [query]
    if language:
         q_parts.append(f"language:{language}")
    if min_stars:
         q_parts.append(f"stars:>={min_stars}")
    q = ' '.join(q_parts)
    results = self.get('/search/repositories', params={'q’: q})
    return results['items']   # items = list of repos
# Later usage: repos = api.search_repos('machine learning', language='python', min_stars=1000)
```

2.7 Reusable API Client — Methods

```python
    def to_dataframe(self, repos):
        """
        Convert list of repo dicts to a clean DataFrame.
        Args:
            repos (list): List of repo dicts from GitHub API
        Returns:
            pd.DataFrame: Structured tabular data
        """
        data = []
        for repo in repos:
            data.append({
                'name'        : repo['name'],
                'full_name'   : repo['full_name'],
                'description': repo.get('description', ''),  # .get() avoids KeyError
                'stars'       : repo['stargazers_count'],
                'forks'       : repo['forks_count'],
                'language'    : repo.get('language', 'Unknown'),
                'created_at' : repo['created_at'],   # ISO 8601 string: '2012-01-01T00:00:00Z'
                'updated_at' : repo['updated_at'],
            })
        return pd.DataFrame(data)
```

2.7 Converting API Data to Pandas DataFrames

```python
# Usage
api   = GitHubAPI(token=os.getenv('GITHUB_TOKEN'))
repo  = api.get_repo('pandas-dev', 'pandas')
repos = api.search_repos('machine learning', language='python', min_stars=1000)
df    = api.to_dataframe(repos)
print(df.head())
print(df.dtypes)

#     name      full_name     description  stars   language  ...
# 0   pandas    pandas-dev... Flexible ... 43256   Python    ...
```

💡 *dict.get('key', default) is safer than dict['key'] — returns default instead of crashing on missing keys.*

✅ *After collecting data: always save immediately with df.to_csv('raw_data.csv', index=False) before processing.*

REST API — Complete Data Collection Workflow

1
Read API Docs
Endpoints, auth, pagination, limits

2
Get API Key
Store in .env, add to .gitignore

3
Test 1 Request
Print response dict, check fields

4
Handle Auth
Add headers with Bearer token

5
Add Pagination
Loop pages until empty response

6
Add Error Handling
Try/except, retry, logging

7
Save Raw Data
df.to_csv('raw.csv') immediately

8
Analyze & Export
Clean, process, visualize, export

REST API Integration — Common Pitfalls & Solutions

❌ **Hardcoding API keys in code**

✅ Always use .env + python-dotenv. Add .env to .gitignore immediately.

❌ **No timeout on requests.get()**

✅ Always set timeout=10. A hanging request blocks your entire script.

❌ **Calling .json() without checking status_code**

✅ Check if response.status_code == 200 first. 404 responses have no JSON.

❌ **Not handling pagination → missing data**

✅ Always loop until you get an empty response. Log total items fetched.

❌ **Ignoring rate limits → 429 errors**

✅ Check X-RateLimit-Remaining header. Add time.sleep() between requests.

❌ **No retry logic → script dies on server error**

✅ Use Retry() from urllib3 or wrap in try/except with exponential backoff.

❌ **Not saving raw data before processing**

✅ Always df.to_csv('raw.csv') immediately after collection. Saves re-fetching.

REST API — Tips, Tricks & Data Science Best Practices

Secrets Safety

Use .env for all keys. Rotate tokens if accidentally committed. Use read-only tokens when possible.

Save Raw Data

Save API responses to disk immediately. Re-fetching wastes rate-limit quota.

Read Docs First

Check rate limits, auth methods, and pagination style before writing a single line of code.

Test with Small Requests

Fetch 1 page before looping. Print one response dict to understand its structure.

Add Polite Delays

Even within rate limits, add time.sleep(0.5) between requests — be a good API citizen.

Convert to DataFrames Early

Convert raw JSON to pd.DataFrame immediately after collection for easy analysis and saving.

Log Everything

Log each request, status code, and items fetched. Logs are priceless when debugging overnight runs.

Validate Data

Check for None values, unexpected types, or missing fields before writing to CSV/database.

⭐ Quick Reference Cheat Sheet — REST API (Part 1)

```python
# ── SETUP ──────────────────────────────────────────────────────
pip install requests python-dotenv                    # Install
import requests, os
from dotenv import load_dotenv
load_dotenv()                                          # Load .env file

# ── BASIC REQUEST ──────────────────────────────────────────────
url      = 'https://api.github.com/repos/owner/repo'
headers  = {'Authorization': f"Bearer {os.getenv('GITHUB_TOKEN')}",
            'Accept': 'application/vnd.github.v3+json'}
params   = {'per_page': 100, 'page': 1}
response = requests.get(url, headers=headers, params=params, timeout=10)

if response.status_code == 200:
    data = response.json()                             # → Python dict

# ── PAGINATION ─────────────────────────────────────────────────
all_items, page = [], 1
while True:
    r    = requests.get(url, params={'page': page, 'per_page': 100})
    data = r.json()
    if not data: break
    all_items.extend(data)
    page += 1;  time.sleep(0.5)

# ── ROBUST SESSION ─────────────────────────────────────────────
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry
session = requests.Session()
retry   = Retry(total=3, backoff_factor=2, status_forcelist=[429,500,502,503])
session.mount("https://", HTTPAdapter(max_retries=retry))
```

⭐ Quick Reference Cheat Sheet — REST API (Part 2)

```python
# ── RATE LIMITING ──────────────────────────────────────────────
remaining = int(response.headers.get('X-RateLimit-Remaining', 999))
if remaining < 10:
    time.sleep(60)                                  # Wait 60s if nearly out

# ── LOGGING ────────────────────────────────────────────────────
import logging
logging.basicConfig(level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[logging.FileHandler('api.log'), logging.StreamHandler()])
logger = logging.getLogger(__name__)
logger.info("Fetching page 1...")

# ── JSON TO DATAFRAME ──────────────────────────────────────────
import pandas as pd
df = pd.DataFrame([{'name': r['name'], 'stars': r['stargazers_count']} for r in repos])
df = pd.json_normalize(nested_json, record_path=['items'], sep='_')
df.to_csv('output.csv', index=False)                  # Save immediately!

# ── STATUS CODES QUICK GUIDE ───────────────────────────────────
# 200 OK          → response.json()         ✓
# 201 Created     → check Location header   ✓
# 400 Bad Request → check your params       ✗
# 401 Unauthorized→ check your API key      ✗
# 404 Not Found   → check URL / endpoint    ✗
# 429 Too Many    → time.sleep(Retry-After) ↻
# 500 Server Err  → retry with backoff      ↻

# ── .env FILE TEMPLATE ─────────────────────────────────────────
# GITHUB_TOKEN=ghp_xxxxxxxxxxxx
# OPENWEATHER_KEY=xxxxxxxxxxxx
```